NotebookLM-Style RAG Application with LangChain and FAISS

Implementation report | October 14, 2026

1. Project Summary

This project is a NotebookLM-style document assistant. A user creates a notebook, uploads documents (PDF, Word, PowerPoint, Excel, CSV, Markdown, text, JSON and images), and then asks questions about them. Every answer is grounded in the uploaded sources, carries numbered citations such as [1], and links back to the exact page, slide or section. The cited sentence is highlighted in a source viewer.

The first version of the retrieval pipeline was written by hand: numpy vectors, manual cosine similarity, raw HTTP calls to OpenAI and hand-built message lists. In this version the whole RAG pipeline was rebuilt on **LangChain**, with **FAISS** as the vector database. The Flask API and the web interface were kept unchanged, so the application behaves the same for the user while the internals now use standard LangChain components.

2. Technology Stack

Layer	Technology
Orchestration	LangChain (langchain-core, langchain-community, langchain-openai, langchain-text-splitters)
Vector database	FAISS (faiss-cpu), saved to disk with LangChain's `save_local`
Embeddings	OpenAI `text-embedding-3-small`, or a free local BGE model through fastembed
Chat model	OpenAI chat model through `ChatOpenAI`, with automatic fallback models
Reranker	Local cross-encoder through fastembed
Backend	Python, Flask
Frontend	Static HTML, CSS and JavaScript (unchanged)

Layer	Technology
Deployment	Docker and docker-compose

3. Architecture

There are two flows: ingestion (when a file is uploaded) and question answering (when the user asks something).

Ingestion

```
File -> Loader (Documents) -> Text splitter (chunks) -> Embeddings -> FAISS
index (saved to disk)
```

Question answering

```
Question -> Guardrails -> FAISS retriever (top 20) -> Reranker (top 8)
         -> ChatPromptTemplate | ChatOpenAI -> Answer -> Citations and
highlights
```

4. Ingestion Pipeline

4.1 Loading files into LangChain Documents (`app/loaders.py`)

Each file is split by its real structure into "units": one per PDF page, per Word heading section, per PowerPoint slide, or per group of 25 spreadsheet rows. Every unit is a LangChain `Document`: the text is in `page_content`, and the metadata holds a label (for example "Page 3" or "Slide 2"), an optional heading, and a flag that lets very small sections merge into the next one.

The parsing itself (PDF text extraction with OCR for scanned pages, DOCX, PPTX, XLSX and so on) is custom code. This was a deliberate choice: it gives accurate page and slide citations and OCR support, which the generic LangChain loaders do not provide. The output is still standard `Document` objects, so everything after this step is plain LangChain.

4.2 Chunking (`app/chunking.py`)

Chunking is done with LangChain's `RecursiveCharacterTextSplitter`:

- chunk size of 1200 characters and an overlap of 200 characters (both configurable);

- it prefers to cut at paragraph breaks, then at sentence ends (including the Urdu full stop), then at word boundaries;
- `add_start_index=True` records where each chunk starts inside its unit, which is what later makes sentence highlighting possible;
- a chunk never crosses a page or section boundary, so a citation label is always correct.

Each chunk stores two texts: the plain chunk text (shown to the user and read by the model) and a "context" text, which is the heading plus the chunk text, used only for embedding.

4.3 Embeddings (`app/embeddings.py`)

Embeddings follow LangChain's `Embeddings` interface (`embed_documents` and `embed_query`). Three implementations exist:

- `OpenAIEmbeddings` (the default in the provided configuration);
- `LocalEmbeddings`, our own `Embeddings` subclass around a free BGE model through fastembed, which adds the query prefix that BGE models require;
- `FakeEmbeddings`, a deterministic hashing embedder used only by the automated tests.

Because FAISS only talks to the `Embeddings` interface, switching providers needs no other code change. A small wrapper returns unit-length vectors, so FAISS inner product equals cosine similarity.

4.4 Vector database (`app/store.py`)

Each notebook has its own **FAISS** index. Every chunk is stored as a LangChain `Document` whose metadata contains the source id, file name, chunk index, unit number, label, original text and start offset. The index is saved to disk with `save_local`, together with a small file recording which embedding model built it.

This gives the following behaviour:

- adding a document embeds its chunks and appends them to the index;
- deleting a document removes exactly its vectors from the index;
- after a restart the index is loaded from disk;
- if the embedding model is changed, the index is detected as outdated and sources are marked "stale" until the user presses **Re-index now**, which re-embeds the stored chunks without re-uploading any file;

- sources saved by the earlier numpy version are migrated the same way.

5. Retrieval

5.1 A LangChain retriever (`NotebookRetriever`)

Retrieval is exposed as a standard `BaseRetriever`. Calling `retriever.invoke("question")` returns the most similar chunks from FAISS by cosine similarity. A metadata filter restricts the search to the sources the user has ticked in the interface, so chunks from unselected documents are never returned.

5.2 Retrieval chain with reranking (`app/rag.py`)

Retrieval is composed with LangChain Expression Language (LCEL):

```
RunnableParallel(query = passthrough, docs = retriever)  |
RunnableLambda(rerank)
```

FAISS first returns the top 20 candidates. A local cross-encoder, implemented as a LangChain `BaseDocumentCompressor` (`app/reranker.py`), then re-scores each candidate against the question and keeps the best 8. If the reranker is unavailable, the application falls back to the vector order and continues to work.

Two extra behaviours improve answer quality:

- **Follow-up questions:** short follow-ups are combined with the previous question before searching, so that pronouns such as "it" are resolved.
- **Broad requests:** requests such as summary, study guide, FAQ, timeline or quiz do not suit similarity search, so evenly spaced chunks are sampled across all selected documents instead.

The model also receives the neighbouring chunk before and after each hit, which fixes explanations that continue across a page break.

6. Answer Generation

6.1 Prompt (`ChatPromptTemplate`)

The prompt has three parts: a system message that contains the numbered passages and the rules (answer only from the passages, cite with [n], and reply in the user's language), a `MessagesPlaceholder` that carries the recent chat history as `HumanMessage` and `AIMessage` objects, and the user's question.

6.2 Model call (`app/llm.py`)

The answer is produced by the chain `prompt | ChatOpenAI`. Features used:

- `ChatOpenAI` with timeout, automatic retries and an output-token limit;
- `.with_fallbacks()`, so that if the main model is unavailable or rejects the request, the configured backup models are tried in order;
- `AIMessage.usage_metadata`, which provides the exact input and output token counts for every call;
- translation of OpenAI errors (invalid key, no credit, rate limit, network) into clear messages shown in the interface.

6.3 Follow-up suggestions

After each answer, a second small chain (a prompt piped into the same model) proposes three follow-up questions shown as clickable cards.

7. Citations and Highlighting

This part is application logic rather than LangChain. The model cites passages as [1], [2] and so on. The server maps each number back to a file and page, and then finds the sentence in the passage that best supports the cited statement, using weighted keyword overlap. The viewer scrolls to that page and highlights the sentence.

A problem appeared when the user wrote in Roman Urdu and the document was in English: the answer and the passage share almost no words, so no sentence could be highlighted. To solve this, a **semantic fallback** was added. When the word match is weak, the answer and the candidate sentences are compared by embedding similarity, which works across languages with OpenAI embeddings. The closest sentence is highlighted

only if it clearly stands out; otherwise nothing is highlighted, which is safer than highlighting the wrong sentence. The small extra embedding cost is added to the cost shown in the interface.

8. Cost Tracking

Every answer shows its own cost: embedding tokens, input tokens, output tokens, total in dollars, response time and the model that answered. The notebook header shows the running total. Chat-model token counts are exact (from `usage_metadata`). Embedding token counts are estimates (about four characters per token), because LangChain embedders do not report usage.

9. Guardrails, API and Notebooks

The Flask API provides notebooks, uploads, chat, suggestions, source viewing and re-indexing. Guardrails run before the model is called: input moderation, request limits and file type checks. Each notebook has its own sources, FAISS index, chat history and cost totals. This layer was not changed during the LangChain migration.

10. Testing and Evaluation

Automated tests (`tests/`): 18 tests run with a fake embedder and a fake LangChain chat model, so they need no API key, no internet and cost nothing. They cover chunking, FAISS add, search, delete and persistence, source filtering, re-indexing after an embedding change, migration of old data, chat history reaching the model, cost calculation, API endpoints and error messages. All 18 pass.

Retrieval evaluation (`eval/`): a script indexes a document, runs a set of questions, and reports **Hit@k** (how often the correct passage appears in the top k) and **MRR** (how high it is ranked), once with vector search only and once with reranking. This measures retrieval quality using the real embedding provider.

11. What Changed in the Migration

Component	Before	After (LangChain)
Text units	Plain dictionaries	`Document`

Component	Before	After (LangChain)
Chunking	Custom function	`RecursiveCharacterTextSplitter`
Embeddings	Custom functions	`Embeddings` interface (OpenAI, local, fake)
Vector storage	numpy arrays and manual cosine search	FAISS vector store
Search	Manual filtering and sorting	`BaseRetriever` with metadata filter
Reranking	Standalone function	`BaseDocumentCompressor`
Pipeline wiring	Manual code	LCEL (`RunnableParallel` , `RunnableLambda` , `|`)
Prompt	String and dict messages	`ChatPromptTemplate` and `MessagesPlaceholder`
LLM call	Raw HTTP requests, manual retry and fallback	`ChatOpenAI` and `.with_fallbacks()`
Token usage	Parsed from raw JSON	`AIMessage.usage_metadata`

Unchanged: the Flask API, the web interface, guardrails, notebook management, cost accounting and the citation and highlight logic.

12. How to Run

1. Copy `.env.example` to `.env` and set `OPENAI_API_KEY` (and `EMBED_PROVIDER=openai`).
2. Start with Docker: `docker compose up --build` , then open `http://localhost:5000` .
3. Run the tests: `python3 -m pytest -q` (expected: 18 passed).
4. Run the retrieval evaluation: `python3 -m eval.evaluate --docs <file.pdf> --dataset eval/sample_dataset.jsonl` .